O-ETB Version 1.x
Design Notes

Rance DeLong*

The Open Group

19 March 2026

Contents

Table of Contents i

List of Figures iv

Executive Summary v

1 Introduction 1

2 ACO Workbench 1

3 APL language 1

4 APL Argument pattern instantiation 1

4.1 Identifier generation and naming during instantiation 1

4.1.1 Unit-scoped identifiers in ACO 1

4.1.2 Translation to patterns 1

4.1.3 Instantiation occurrences and instance identifiers 1

4.1.4 Lifting unit-scoped IDs into case-wide scope 2

4.1.5 Multiple instantiations and repeated calls 2

4.1.6 User-visible behavior 3

5 Case Instantiation Order and Export Semantics 3

5.1 Repository-scoped case model 3

*r.delong@opengroup.org

5.2 Instantiation order as semantic structure . . . 3
5.3 Export semantics derived from this model . . . 4
5.4 Diagnostic instances and completeness . . . 4
5.5 Design principle . . . 5

6 Resets **5**

7 Shared Argument Reuse and `ac_shared_goal_ref` **6**
7.1 Intentional Sharing vs. Accidental Duplication . . . 6
7.2 Semantic Nature of Shared Goals . . . 7
7.3 Reuse Scope and Validity . . . 7
7.4 Post-hoc Identification of Sharing Opportunities . . . 7
7.5 Relationship to Evidence Execution . . . 7
7.6 Summary . . . 7

8 Intentional Sharing of Argument Instantiations **8**
8.1 Motivation . . . 8
8.2 Conceptual Model . . . 8
8.3 Design Objectives . . . 9
8.4 Key Design Decision . . . 9
8.5 Authoring Model . . . 9
8.6 Instantiation Semantics (Deferred) . . . 10
8.7 Export and Diagnostics Implications . . . 10
8.8 Trade-offs and Deferred Work . . . 10
8.9 Status . . . 11

9 ACO Transformations **11**

10 AI-based 6-Step Method Agent **11**
10.1 Implementing a tool within a general AI chat bot . . . 11
10.2 Motivation for the approach . . . 11
10.3 Implementation alternatives . . . 12
10.4 Chosen implementation approach . . . 12
10.5 Packaging the Kelly 6-Step Assistant as an Agent Skill . . . 12
10.6 Conceptual Architecture . . . 13
10.6.1 Layer 1: Method Layer — Kelly 6-Step Assistant (Agent Skill) . . . 13
10.6.2 Layer 2: Transformation Layer — ACO Workbench . . . 14
10.6.3 Layer 3: Execution and Evidence Layer — O-ETB Core . . . 14

10.6.4 Separation of Concerns . . . 15

11 ACO Transformations (T1–T7) Formal Definition 15

11.1 ACO Structural Model . . . 15
11.2 Transformation Framework . . . 16
11.3 T1 — Evidence Node Slimming . . . 17
11.4 T2 — Evidence-Discharging Goal Introduction . . . 17
11.5 T3 — Artifact Role Reclassification . . . 18
11.6 T4 — Evidence Category Grounding . . . 18
11.7 T5 — Basis Articulation (Kelly Steps 2 and 4) . . . 19
11.8 T6 — Goal-to-Module Modularization . . . 19
11.9 T7 — Strategy Insertion . . . 20
11.10 Deterministic Candidate Generation . . . 21
11.11 Implementation Architecture . . . 21
11.12 Ordering Strategy . . . 21
11.13 Correctness Constraints . . . 22
11.14 Summary . . . 22

12 Deterministic Candidate Ranking and Body Text Synthesis 22

12.1 Observe–Rank–Transform workflow . . . 22
12.2 Kelly deficit scoring . . . 23
12.3 `AUTHOR CHECK:` convention and conservative synthesis . . . 23
12.4 Relationship to bounded LLM refactoring . . . 24

13 Bounded LLM-Assisted Semantic Refactoring 24

13.1 Rationale . . . 24
13.2 Architectural Principle . . . 24
13.3 Formal Refactoring Contract . . . 25
13.4 Traceability Constraint . . . 25
13.5 Transformation-Specific Contracts . . . 26
13.6 Determinism and Versioning . . . 26
13.7 Fallback Mode . . . 27
13.8 Separation of Concerns . . . 27

14 Action Layer and Constructor Basis 28

14.1 Canonical action representation . . . 28
14.2 Candidates, observation, and ranking . . . 28
14.3 Constructor basis for assurance case construction . . . 28

14.4 Targets and goal nesting 29
14.5 Diagnostics and reporting 29
14.6 Relation to GSN and APL completeness 30

15 Iterator Validation Boundaries and Extensible Iterand Semantics 30
15.1 Motivation 30
15.2 Three distinct notions of well-formedness 31
15.3 Design consequence for ACO 31
15.4 Architectural implication: an O-ETB validation API 31
15.5 Extensible provider architecture 32
15.6 Implementation guidance 32
15.7 Status 33

List of Figures

1 Layered architecture for method-driven assurance case refinement and execution. 15

Executive Summary

This work describes the co-evolution of a real assurance case for the Remote Patient Monitoring (RPM) use case and the structured tooling required to refine it in a disciplined, repeatable manner.

The effort has resulted in a three-layer architecture separating method, structural transformation, and execution concerns.

Objective

The objective is to enable method-driven, reproducible refinement of assurance cases using Kelly's 6-Step Method, while preserving author intent and maintaining deterministic tooling behavior.

Layered Architecture

Method Layer The Kelly 6-Step Assistant encodes refinement discipline as a stateful process model. It identifies deficiencies (particularly Step 2 and Step 4 issues), recommends structural transformations (T1–T7), and prioritizes remediation using rule-based scoring. It is packaged as an Agent Skill to ensure portability and reproducibility.

Transformation Layer The ACO Workbench provides deterministic structural transformations over Assurance Case Outline artifacts. Transformations include evidence slimming (T1), introduction of dischargeable goals (T2), artifact role clarification (T3), evidence grounding (T4), basis articulation (T5), modularization (T6), and strategy insertion (T7). All transformations preserve semantics and follow minimal-diff discipline.

Execution and Evidence Layer The O-ETB core provides pattern instantiation, tool orchestration, provenance management, and CAP export. REPOSITORY remains the authoritative source of run records; CAP provides evaluator-facing snapshots. Optional MCP integration enables standardized tool exposure.

Demonstrated Through RPM Use Case

The RPM assurance case (formerly BioAssist) served as the proving ground for:

- Identifying recurring structural deficiencies,
- Defining transformation families,
- Refining deterministic ACO processing,
- Demonstrating modularization at scale,
- Developing a method-encoded refinement assistant.

Exemplars (E1122, E1121, E511) illustrate the transition from plausible argument structure to method-disciplined, reviewer-aligned assurance cases.

Contribution

The contribution extends beyond tooling:

1. Formalization of assurance-case refinement as a structured transformation system,
2. Encoding of Kelly's method as an enforceable state model,
3. Separation of structural transformation from execution and provenance,
4. Packaging of methodological discipline as a portable Agent Skill.

This architecture enables repeatable, scalable, and auditable development of assurance cases in safety- and security-critical domains.

1 Introduction

2 ACO Workbench

3 APL language

4 APL Argument pattern instantiation

4.1 Identifier generation and naming during instantiation

This section describes how identifiers (IDs) appearing in Assurance Case Outlines (ACOs) are interpreted, transformed, and generated during pattern instantiation in O-ETB. The intent is to ensure that identifiers remain readable and traceable for users, while being unambiguous, stable, and suitable for repeated instantiation, transformation, and incremental re-execution by the tool.

4.1.1 Unit-scoped identifiers in ACO

In the ACO language, identifiers declared in *node headers* (e.g., `G0`, `S1`, `E10`, `M3`) are required to be unique only within the enclosing *unit*. A unit is introduced by a `Case:` or `Module:` unit header.

This unit-local scoping rule mirrors common authoring practice and aligns with the one-to-one correspondence between ACO and the industry-standard Goal Structuring Notation (GSN), in which identifiers are primarily used for local reference and human readability rather than global uniqueness.

Identifiers declared in different units may therefore legally coincide.

4.1.2 Translation to patterns

Before instantiation occurs, each ACO unit is translated into an APL pattern. At this stage, identifiers are preserved exactly as authored. No attempt is made to enforce case-wide uniqueness during translation.

For patterns authored directly in APL (without an ACO precursor), identifiers are taken exactly as provided by the pattern author and are treated in the same manner during instantiation.

4.1.3 Instantiation occurrences and instance identifiers

Pattern instantiation is performed by the O-ETB instantiator, which interprets patterns as executable specifications that incrementally construct an instantiated assurance case in the REPOSITORY.

Each time a pattern reference is instantiated, a distinct *instantiation occurrence* is created. An instantiation occurrence corresponds to a specific execution step of the instantiator and is defined by:

- the parent instantiation occurrence in which the call appears,
- the relative structural position of the call within the parent pattern,
- and the dynamic execution index of that call at that position.

For each instantiation occurrence, O-ETB allocates a fresh *instance identifier* (`InstId`). Instance identifiers are opaque positive integers that are unique within the scope of a single instantiated assurance case.

4.1.4 Lifting unit-scoped IDs into case-wide scope

When a pattern is instantiated, all node identifiers originating from that pattern (whether translated from ACO or authored directly in APL) are lifted from unit scope into case-wide scope by deriving new identifiers that incorporate the instance identifier of the instantiation occurrence.

Conceptually, each node identifier is transformed as follows:

$$NewId = BaseId_InstId$$

where:

- *BaseId* is the identifier as authored in the ACO or APL pattern, and
- *InstId* is the instance identifier allocated for the instantiation occurrence.

All nodes originating from the same instantiation occurrence share the same instance identifier suffix. This guarantees:

- global uniqueness of identifiers in the instantiated case,
- clear traceability from instantiated nodes back to their originating unit and occurrence,
- and stable naming across repeated executions of the instantiator, provided the instantiation structure is unchanged.

4.1.5 Multiple instantiations and repeated calls

If the same pattern is instantiated multiple times (for example, due to repeated module references, iteration constructs, or recursive use of patterns), each instantiation occurrence receives its own distinct instance identifier, even if the pattern identifier and actual arguments are identical.

This ensures that structurally distinct parts of the instantiated assurance case are never conflated, while still allowing the tool to recognize opportunities for reuse and caching internally based on richer occurrence metadata.

4.1.6 User-visible behavior

The identifiers produced by this scheme are the identifiers that appear in exported assurance cases (e.g., textual or HTML representations in CAP). Users are not required to provide any explicit disambiguation of identifiers beyond the unit-scoping rules of ACO.

The instance identifier suffixes are generated entirely by the tool and reflect the structure of the instantiated assurance case. Their purpose is to ensure correctness, traceability, and efficient incremental processing, rather than to be manually authored or interpreted by users.

5 Case Instantiation Order and Export Semantics

5.1 Repository-scoped case model

In O-ETB, a *case* is represented by an attached CASES repository, selected via:

```
current_assurance_repository/1
```

At any time, exactly one CASES repository is active, and it contains *only* the instances belonging to the current case. No instances from other cases are present.

Case scoping is therefore achieved by *repository attachment*, not by filtering within a shared global store.

5.2 Instantiation order as semantic structure

During instantiation, the ETB instantiator dynamically interprets APL patterns starting from a designated root (the "trunk"). As patterns invoke other patterns, instances are created and asserted in the CASES repository.

Key property

> **The `ac_instance/5` facts are asserted in the exact order in which instantiation occurs.**

This order has semantic meaning:

1. The **first asserted instance** is the **root** of the instantiated assurance case.
2. Subsequent instances reflect the **dynamic execution trace** of the instantiator:
   - parent patterns first,
   - then their children,
   - then grandchildren, and so on.
3. The repository therefore contains a **linearization of the construction of the case tree**.

No additional indexing or graph reconstruction is required to identify either the case root or the causal order of instantiation.

5.3 Export semantics derived from this model

Given this model, the correct export semantics are:

1. `export_case/2` operates on the **currently attached CASES repository**.
2. All `ac_instance/5` facts in that repository belong to the case.
3. The **first instance** is treated as the **root** of the case.
4. Export proceeds by:
   - rendering the root instance as the main case artifact, and
   - rendering remaining instances as auxiliary artifacts (e.g., per-instance HTML/DOT files).

This design deliberately avoids:

- PatternId-based filtering,
- subtree reconstruction,
- ad-hoc graph traversal during export.

Instead, export relies on two stable invariants:

- **repository scoping**, and
- **assertion order**.

5.4 Diagnostic instances and completeness

Instances created for diagnostic purposes (e.g., null instances for missing patterns) are legitimate members of the CASES repository. Their presence:

- supports traceability and debugging, and
- preserves a faithful record of the instantiation process.

Whether such instances are shown in user-level exports is a **presentation policy**, not a repository or instantiation concern.

5.5 Design principle

This approach embodies a central ETB design principle:

> **Execution order is semantic data.** The instantiator's dynamic behavior is not merely procedural; it is part of the assurance evidence and must be preserved, not normalized away.

6 Resets

A reset command is executed as follows:

- The O-ETB command *reset.* is first recognized by the command module which passes it on as *etb* : *etb_reset*/0 defined in the *etb* module.
- *etb* : *etb_reset* calls:

  $$evidence : detach_evidence_repository$$

  and

  $$assurance : detach_assurance_repository$$

  to assure that the in-memory persistent data is flushed.
- *etb* : *etb_reset* does a complete reset simply by invoking

  $$make\ \text{-}s\ clean$$

  using the *makefile* definition of the *clean* target to remove and initialize files in both:

  $$REPOSITORY/CASES$$

  and

  $$REPOSITORY/EVIDENCE$$

  directories (currently no action is taken on $REPOSITORY/OUTLINES$).
- After the *make clean* operation the evidence repository is reattached with

  $$attach_evidence_repository$$

  since there is only one evidence repository that should always be accessible to O-ETB.

There are also entry points in module *etb* for *etb_reset*/1 with the argument taking the possible values: *all*, *cap*, *repos*, *cases*, and *evidence*. The different values for the argument have the following effects:

- *all* - causes *etb_reset*/0 to be invoked.
- *cap* - causes *export* : *reset_CAP*/0 to be invoked.

- *repos* - causes the invocation of:

$$etb_reset(cases)$$

  and

$$etb_reset(evidence)$$

  to be invoked.

- *cases* - causes $assurance : reset_assurance_repository$ to be invoked.
- *evidence* - causes $evidence : reset_evidence_repository$ to be invoked.

The predicate $assurance : reset_assurance_repository$ uses "make" with the target *clean_cases after* calling:

$$assurance : detach_assurance_repository$$

The predicate $evidence : reset_evidence_repository$ uses "make" with the target *clean_evidence after* calling:

$$evidence : detach_evidence_repository$$

To restore access to the evidence repository it also invokes:

$$evidence : attach_evidence_repository$$

after calling *make*.

The predicate $export : reset_CAP$ uses *make* with the target *clean_cap*.

The use of *make* from within O-ETB process, while creating an external dependency, does guarantee consistency of the reset whether it is performed in O-ETB or outside the application as a *make* command by having the *makefile* provide the sole definition of the various *clean* operations.

7 Shared Argument Reuse and `ac_shared_goal_ref`

Large assurance cases frequently contain arguments that are intentionally relied upon in more than one place. In such situations, duplicating the same argument structure and evidence obscures author intent, complicates maintenance, and risks inconsistency. To address this, O-ETB supports explicit argument sharing via the `ac_shared_goal_ref` construct.

7.1 Intentional Sharing vs. Accidental Duplication

Sharing in O-ETB is an explicit authoring decision. It denotes that the *same assurance claim*, supported by the *same justification and evidence*, is intentionally relied upon in multiple locations in the case. This is distinct from accidental duplication, where similar argument structures arise independently without an explicit commitment to identity.

The use of `ac_shared_goal_ref` establishes such intentional identity.

7.2 Semantic Nature of Shared Goals

The `ac_shared_goal_ref` construct represents an *equivalence (identity) relation* over assurance arguments. It asserts that multiple occurrences in the argument structure refer to a single shared argument instance. This identity relation is distinct from dependency relations such as `supports` or `in_context_of`.

While identity entails shared dependency relationships (e.g., the shared argument supports each of its parent goals), `ac_shared_goal_ref` does not itself assert a dependency. Rather, dependency edges are induced by the placement of the shared reference within the parent argument structures.

7.3 Reuse Scope and Validity

A shared argument instance must be valid in each parent context in which it is referenced. Importantly, this does not require the parent contexts to be identical. Instead, it requires that:

> All assumptions and dependencies of the shared argument are satisfied in each parent context.

To support this, each shared goal is associated with a *reuse key*, defined as the minimal set of properties that must be identical for the claim and its supporting evidence to remain valid. Typical reuse keys are based on contracts, certified artifacts, qualified tools or processes, or platform configurations, together with their versions.

7.4 Post-hoc Identification of Sharing Opportunities

Opportunities for sharing are often discovered during case evolution or review, rather than during initial authoring. O-ETB therefore permits post-hoc transformation of independently instantiated argument subtrees into shared goals. This supports realistic development workflows in which different authors contribute independently and harmonization occurs later.

7.5 Relationship to Evidence Execution

Shared argument identity entails a single logical evidence instance within a case. This is complementary to evidence idempotency mechanisms, which ensure that evidence production processes are not re-executed unnecessarily across repeated runs when inputs have not changed.

7.6 Summary

In summary, `ac_shared_goal_ref` denotes intentional reuse of a single concrete argument subtree. The first occurrence defines the shared subtree, and all subsequent occurrences are non-instantiating aliases that rely on that same argument subtree. It improves clarity,

maintainability, and analytical soundness by making identity explicit, while preserving flexibility through well-defined reuse scopes.

8 Intentional Sharing of Argument Instantiations

8.1 Motivation

During the evolution of large assurance cases, it is common for equivalent or identical argument structures to arise independently in different branches of the case. This duplication is often not intentional at first and may only be recognized during later review or consolidation. When left unaddressed, such duplication can lead to:

- redundant instantiation of argument structures,
- repeated execution or collection of the same evidence,
- unnecessary growth in case size and complexity, and
- reviewer confusion when multiple arguments appear distinct but are, in fact, intended to be the same.

The goal of intentional sharing is to allow an author to explicitly state that multiple parts of an assurance case rely on the *same* argument, supported by the same justification and evidence, without duplicating that argument structure.

8.2 Conceptual Model

An intentionally shared argument is not a new ontological entity. Fundamentally, it is a concrete instantiation of an existing argument pattern with a particular set of bound arguments. Sharing does not introduce a new kind of argument; rather, it introduces a way to *reuse the identity* of an already-instantiated argument subtree.

Thus, a shared argument:

- is produced by pattern instantiation,
- has a single concrete goal/strategy/evidence subtree,
- may be referenced from multiple parent goals, and
- enforces single-execution semantics for its evidence.

This distinguishes intentional sharing from:

- accidental duplication (multiple independent instantiations), and
- explicit relation sentences (e.g., `supports`, `in_context_of`), which assert semantic relationships but do not establish identity.

8.3 Design Objectives

The design of intentional sharing is guided by the following objectives:

- Preserve the existing pattern instantiation model and execution semantics.
- Avoid introducing a second, parallel instantiation mechanism.
- Keep the internal vocabulary minimal and conceptually clean.
- Maintain precise occurrence identity for navigation, diagnostics, and traceability.
- Adhere to the Principle of Least Astonishment: the system should behave as a knowledgeable assurance author would expect based on the artifacts they authored.

8.4 Key Design Decision

The central design decision is that intentional sharing is modeled as:

> *A pattern instantiation with an additional sharing attribute and associated identity registry.*

Operationally:

- The first occurrence of a shared argument causes instantiation of the pattern.
- Subsequent occurrences do not instantiate; they act as aliases to the same concrete subtree.
- Each occurrence retains its own occurrence identity and parent context.

This approach treats sharing as an optimization over multiple instantiations of the same pattern with the same arguments, rather than as a distinct construction mechanism.

8.5 Authoring Model

At the ACO level, intentional sharing is expressed tersely, consistent with the outline style:

```
shared(Name, Args)
```

The mapping from a shared name to the underlying pattern is declared explicitly in the same ACO unit:

```
shared_def(Name, PatternId)
```

This declaration-based approach ensures that:

- sharing intent is author-controlled and explicit,
- the mapping travels with the ACO artifact, and
- sharing can be introduced incrementally as the case evolves.

Use of `shared/2` without a corresponding declaration is permitted during authoring and translation but must be resolved before instantiation.

8.6 Instantiation Semantics (Deferred)

During instantiation, shared arguments require a global registry for the duration of a run. Conceptually, this registry maps a *sharing key* (derived from the shared name and bound arguments) to a single concrete instantiation.

The instantiation semantics are as follows:

- The first encountered occurrence of a given sharing key defines the shared argument instance.
- All subsequent occurrences become non-instantiating aliases.
- Each alias induces an ordinary support relationship from the shared argument root to its parent.

Occurrence identity and instantiation identity remain distinct.

8.7 Export and Diagnostics Implications

Shared arguments must be clearly distinguishable in exports and diagnostics:

- Shared references should be visibly marked.
- Unmapped or unresolved shared references must be reported explicitly.
- No silent inference of sharing is permitted.

Presentation layers may choose compact or expanded views, but the underlying identity semantics remain intact.

8.8 Trade-offs and Deferred Work

While the design is conceptually clean, its implementation is non-trivial. It requires:

- careful handling of occurrence ordering,
- interaction with iterators and execution paths,
- robust diagnostics, and
- a suite of automatable regression tests.

Given current project priorities, particularly the development and leadership of partner assurance cases, implementation of intentional sharing is deferred.

8.9 Status

The design for intentional sharing of argument instantiations is complete and recorded here. Implementation is intentionally postponed until adequate development time and test infrastructure are available.

9 ACO Transformations

This material has been merged into Section 11 as an overview to the formal transformation definitions.

10 AI-based 6-Step Method Agent

A need arose while working on the Remote Patient Monitoring (RPM) use case as we sought to apply Kelly's Six-Step Method to develop and mature the assurance case.

10.1 Implementing a tool within a general AI chat bot

We are assembling a paste-in initialization record for an AI chat bot (currently using ChatGPT 5.2) to get it to act as a "Kelly's 6-Step Assistant Tool" that could be used in conjunction with the ACO Workbench (the new O-ETB subsystem that provides assurance case outline (ACO) functionality). My idea was to provide a package that anyone with access to ChatGPT could use to initialize a fresh instance of ChatGPT on the user's own account to recreate an AC evaluator and transformation recommender "tool." (Of course other AI agents could be used in the future, but we would initially use only a single AI agent to limit the amount of testing we would have to do, and to achieve consistent results.) This "tool" would be treated as part of the ACO extensions to O-ETB.

10.2 Motivation for the approach

This idea emerged while contemplating how to extend the simple syntax and semantic checks that were already being done in the O-ETB/ACO extension to provide "more intelligent" recommendations that would help the AC author to follow the 6-Step method. Beyond merely indicating the applicability of structural transformations, by applying well-defined ACO syntactic structures, and objective rules, it is necessary for the system to *understand* what the NL text in the AC is saying to assess whether the objectives of the steps are being achieved and what changes are necessary to effect those steps. The extensions would couple AC assessment metrics computations with AC transformations that would yield new AC states that incrementally improve the metric assessment.

10.3 Implementation alternatives

The problem perceived is that while achieving this deeper "understanding" is possible algorithmically, it would be a torturous path to follow to incrementally develop and refine the needed rules and algorithms. What would be ideal is to have AI-based NL understanding *within* the ACO Workbench to use in conjunctions with rules to drive the 6-Step method, metrics, and transformations. The interim approach to achieve the NL understanding capabilities "within" the Workbench was to factor-out an AI-based tool that would understand ACs, our representations, transformations and metrics available in the Workbench, the GSN standard and Kelly's method. Such a tool could be used in conjunction with an O-ETB/ACO session or standalone.

10.4 Chosen implementation approach

The problem is reduced to getting an AI chat bot, which is capable of NL processing, to *act* like a *tool* rather than like an unpredictable probabilistic generator. We've been proceeding by defining the expected external tool function and providing a package of instructions and knowledge resources that a user could use to initialize a ChatGPT session and to have it review your AC and give suggestions on transformations to apply, guided by the 6-Step Method, that would shepherd the AC into a better state. After skimming the reference (https://agentskills.io/home) it seems to me as though our effort may be very similar to the "AI skills" concept referenced, and we look forward to learning more about it, and if our beliefs are borne out, learning from it and aligning our future efforts with it.

10.5 Packaging the Kelly 6-Step Assistant as an Agent Skill

The development of the Kelly 6-Step Assistant Tool has, until now, been framed primarily as an internal methodological extension of the ACO Workbench. However, the emerging *Agent Skills* standard provides a natural and effective packaging and distribution mechanism for this work.

The Kelly Assistant is designed to act as a deterministic assurance-case analysis tool. Its purpose is to encode and operationalize:

- The Kelly 6-Step Method as a stateful process model,
- Explicit identification of Step 2 (claim basis) and Step 4 (strategy basis) deficiencies,
- A catalog of mechanical ACO transformations (T1–T7),
- Structured diagnostic vocabulary and repeatable output templates,
- Priority scoring to guide refinement effort.

In practical terms, the Assistant can be packaged as an Agent Skill containing:

- A `SKILL.md` defining operating discipline, step-transition rules, and output formats,

- Normative references (Kelly 1998/2001; Hawkins–Kelly 2009; GSN Standard; ACO specification),
- The transformation catalog and recognition heuristics,
- Worked exemplars (e.g., E1122, E1121, E511),
- Defined diagnostic outputs such as

$$(\text{NodeID}, \text{PriorityScore}, \text{KellyStepToStartWith}, \text{SuggestedTransformationFamily}).$$

Adopting the Agent Skills format does not alter the intellectual core of the work; rather, it provides a standardized container for its distribution and reuse. The contribution extends beyond domain-specific prompting. It encodes:

1. A step-constrained refinement process (enforcing valid Kelly step transitions),
2. A transformation algebra over ACO artifacts,
3. Metric-guided improvement of assurance-case structure and sufficiency.

In this sense, the Skill format serves as the packaging layer, while the substantive contribution lies in formalizing the assurance-case refinement method itself. Future integration with an MCP server could expose executable ACO operations, with the Skill orchestrating method-compliant analysis and transformation recommendations.

This approach allows the Kelly 6-Step Assistant to be reproducible, shareable, and extensible, while preserving methodological rigor and alignment with established assurance standards.

10.6 Conceptual Architecture

The proposed architecture separates method, transformation, and execution concerns into three logically distinct layers (Figure 1). This separation is deliberate and reflects the need for determinism, auditability, and disciplined refinement of assurance cases.

10.6.1 Layer 1: Method Layer — Kelly 6-Step Assistant (Agent Skill)

The top layer encodes the refinement method. It is implemented as a packaged Agent Skill that causes a general-purpose AI agent to behave as a deterministic assurance-case analysis tool.

This layer is responsible for:

- Enforcing Kelly's 6-Step process as a stateful refinement model,
- Identifying Step 2 (claim basis) and Step 4 (strategy basis) deficiencies,
- Applying a controlled diagnostic vocabulary,
- Producing repeatable analysis outputs,

- Recommending specific structural transformations (T1–T7),
- Prioritizing remediation using rule-based scoring.

The Method Layer does not mutate artifacts directly. It analyzes ACO-visible structures and produces transformation recommendations in a structured, deterministic form. It operates primarily over Assurance Case Outline (ACO) artifacts and does not depend on internal REPOSITORY state.

10.6.2 Layer 2: Transformation Layer — ACO Workbench

The middle layer implements structural transformations that are deterministic and mechanically applied over ACO artifacts.

It is realized as an extension to O-ETB providing structured ACO processing capabilities. Natural-language rewriting required by these transformations is generated either by deterministic templates or, optionally, by bounded LLM-assisted refinement that is restricted to designated node bodies and must provide traceability to the original text.

Responsibilities include:

- Parsing and canonicalizing ACO source,
- Applying structural transformations (T1–T7),
- Performing modularization (T6) with deterministic ordering,
- Maintaining minimal-diff discipline,
- Generating outline/skeleton views to support recognition,
- Translating ACO to APL for instantiation.

This layer enforces structural correctness while preserving author intent. It performs no semantic reinterpretation beyond the explicit transformation definitions.

10.6.3 Layer 3: Execution and Evidence Layer — O-ETB Core

The bottom layer provides execution, orchestration, and provenance management.

Its responsibilities include:

- Pattern instantiation,
- Tool orchestration and execution,
- Authoritative run-record storage in REPOSITORY,
- Evidence artifact management,
- Export to CAP for evaluator consumption,
- Optional exposure of operations via MCP.

REPOSITORY remains the authoritative source of provenance. CAP is a snapshot export intended for evaluator-facing inspection and is intentionally isolated from mutable repository state.

10.6.4 Separation of Concerns

The layered structure ensures:

- Method discipline is encoded independently of execution mechanisms,
- Structural transformations are deterministic and auditable,
- Tool execution and provenance management remain authoritative and reproducible,
- Evaluator-facing artifacts are stable snapshots rather than live views of mutable repository state.

This separation supports the Principle of Least Astonishment[1]: authors reason about ACO artifacts; transformations preserve intent; execution layers do not introduce hidden semantic

Layer 1: Method Layer Kelly 6-Step Assistant (Agent Skill) Step discipline, scoring, T1–T7 recommendations
⇓ Diagnose / Recommend
Layer 2: Transformation Layer ACO Workbench (O-ETB Subsystem) ACO parsing, transformations, modularization, outline views
⇓ Instantiate / Execute
Layer 3: Execution & Evidence Layer O-ETB Core + REPOSITORY + CAP (+ optional MCP) Pattern instantiation, tool runs, provenance, export

Figure 1: Layered architecture for method-driven assurance case refinement and execution.

11 ACO Transformations (T1–T7) Formal Definition

11.1 ACO Structural Model

An Assurance Case Outline (ACO) unit is modeled as a finite, rooted, ordered forest:

$$U = (N, E, r)$$

where:

[1]Daniel J. Edwards - A human-computer interaction should cause as little astonishment as possible to the human.

- N is a finite set of nodes,
- $E \subseteq N \times N$ is the parent–child relation,
- $r \in N$ is the unit root (for `Case:` or `Module:`),
- the induced graph is acyclic and ordered.

Each node $n \in N$ is a typed record:

$$n = \langle type(n), id(n), label(n), body(n), kids(n) \rangle$$

with:

$$type(n) \in \{\mathsf{Goal}, \mathsf{Strategy}, \mathsf{Evidence}, \mathsf{Context}, \mathsf{Justification}, \mathsf{Assumption}, \mathsf{ModuleRef}\}$$

Let:

- $parent(n)$ denote the unique parent of n,
- $subtree(n)$ denote all descendants of n including n,
- $depth(n)$ denote its indentation depth,
- $pos(n)$ denote its structural path within the unit.

Transformations operate at the ACO level only and must preserve:

- acyclicity,
- sibling order (except where explicitly rewritten),
- node identifiers (unless creating new nodes),
- minimal-diff discipline.

11.2 Transformation Framework

Each transformation T_i is defined as a partial function:

$$T_i : U \rightharpoonup U'$$

with structure:

$$\frac{Obs_{T_i}(U, \theta) \quad Sel_{T_i}(U, \theta) = P}{Rewrite_{T_i}(U, \theta, P) = U'}$$

where:

- Obs_{T_i} is a deterministic suitability predicate,
- Sel_{T_i} selects participating nodes,
- $Rewrite_{T_i}$ performs a mechanical tree rewrite,
- θ is a binding environment for identifiers and extracted text.

Recognition and rewrite are strictly separated to ensure determinism and auditability.

11.3 T1 — Evidence Node Slimming

Intent Separate artifact reference from method and adequacy discourse.

Observation For e with $type(e) = \mathsf{Evidence}$:

$$Obs_{T1}(e) \iff \begin{cases} |body(e)| > L_1 \\ \text{OR multiple discourse roles detected} \end{cases}$$

Discourse roles are detected via token classes:

$$\mathcal{R}_{method}, \mathcal{R}_{result}, \mathcal{R}_{adequacy}$$

Rewrite Replace e with:

$$e' = \langle \mathsf{Evidence}, id(e), label(e), ArtifactRef(body(e)) \rangle$$

Introduce optional nodes:

$$c = \langle \mathsf{Context}, id_c, _, MethodText \rangle$$
$$j = \langle \mathsf{Justification}, id_j, _, AdequacyText \rangle$$

Inserted adjacent to e' under $parent(e)$.

Effect Evidence becomes an artifact anchor; sufficiency premises become explicit.

11.4 T2 — Evidence-Discharging Goal Introduction

Intent Eliminate abstraction gaps between abstract goals and concrete evidence.

Observation Given e child of goal g:

$$Obs_{T2}(g, e) \iff Abstract(g) \wedge Concrete(e)$$

where abstraction and concreteness are detected via lexical classes.

Rewrite Introduce intermediate goal g':

$$g' = \langle \mathsf{Goal}, id_{g'}, label', body' \rangle$$

Rewire:

$$kids(g) := (kids(g) \setminus \{e\}) \cup \{g'\}$$
$$kids(g') := \{e\}$$

Effect Evidence directly discharges its immediate parent goal.

11.5 T3 — Artifact Role Reclassification

Intent Distinguish system evidence from argument-construction artifacts.

Observation

$$Obs_{T3}(g, e) \iff ArgumentArtifact(e) \wedge \neg SystemArtifact(e)$$

Rewrite Split e (as in T1), relocate argument-specific material to:

$$c = \langle \mathsf{Context}, id_c, _, ArgumentProcessText \rangle$$

attached to nearest strategy ancestor.

Effect Preserves role purity and prevents implicit semantic strengthening.

11.6 T4 — Evidence Category Grounding

Intent Make evidence categories operationally meaningful.

Observation

$$Obs_{T4}(e) \iff Category(e) \wedge \neg FullyGrounded(e)$$

Rewrite Introduce context node:

$$c_{cat} = \langle \mathsf{Context}, id_c, _, Category \Rightarrow ArtifactForm \Rightarrow Tool \Rightarrow Agent/Run \rangle$$

Effect Supports auditability and automation hooks.

11.7 T5 — Basis Articulation (Kelly Steps 2 and 4)

T5a — Claim Basis

$$Obs_{T5a}(g) \iff MissingStep2Basis(g)$$

Insert:

$$c_{basis} = \langle \mathsf{Context}, id_c, _, Scope/Definitions/Assumptions \rangle$$

T5b — Strategy Basis

$$Obs_{T5b}(s) \iff MissingStep4Basis(s)$$

Insert:

$$j_{basis} = \langle \mathsf{Justification}, id_j, _, Coverage/CompletenessRationale \rangle$$

Effect Externalizes semantic and sufficiency premises.

11.8 T6 — Goal-to-Module Modularization

Overview As assurance cases grow in size, a single monolithic argument structure becomes increasingly difficult to review, navigate, and evolve. Transformation T6 introduces a structural mechanism for decomposing an assurance case into explicit modules, while preserving the original argument semantics.

T6 operates by converting selected goal subtrees into ACO modules and replacing the original goal nodes with module reference nodes. This transformation is purely structural: it does not alter the claims being made, the strategies used, or the evidence cited. Its purpose is to improve scalability, locality of reasoning, and reviewability.

Unit-scoped semantics Modularization is defined as a unit-scoped operation. A T6 transformation applies within a specific ACO unit (either the Case unit or a named Module unit), rather than implicitly ranging over the entire assurance case. This reflects the author's workflow: modularization is typically performed while working within a particular unit that has grown too large to manage comfortably.

Accordingly, O-ETB provides unit-scoped modularization commands, such as modularizing goals within the Case unit or within a specific module unit.

Batch modularization and ordering When T6 is applied to a list of goals within a unit, the transformation is performed in descendant-first (postorder) order with respect to the containment hierarchy of those goals in the unit. This ordering is required to ensure correctness: if an ancestor goal were modularized first, its descendant goals would no longer be present in the unit and could not be modularized as part of the same batch operation.

This ordering rule applies only within a single batch modularization command. Across separate commands or authoring sessions, no global ordering constraint is imposed.

Module references and definitions At the original location of a modularized goal, T6 introduces a module reference node using the ACO module invocation syntax. The module reference explicitly denotes an invocation, even when no parameters are present, to avoid ambiguity.

A corresponding module definition is introduced as a new Module unit. The root of the module is the original goal node, including its full subtree. The goal's identity and semantics are preserved; only its structural location changes.

Rationale By making modularization explicit and mechanical, T6 supports incremental restructuring of assurance cases without semantic drift. It enables authors to begin with a top-level modularization of the case and later modularize deeper substructures within newly created modules as the argument evolves.

Rewrite This transformation complements the other semantic and evidential transformations (T1–T5, T7) by addressing structural scale rather than argumentative content. Given selected goal roots $G = \{g_1, \ldots, g_m\}$:

$$\text{Apply in descending } depth(g_i)$$

- Replace subtree(g) with a ModuleRef node,
- Create Module unit containing subtree(g),
- Preserve $id(g)$ within module scope.

Effect Creates a new Module unit. Replaces relocated subtree with a module reference.

11.9 T7 — Strategy Insertion

Intent Ensure that each non-leaf goal has an explicit strategy node providing a stable location for Kelly Step 3 (strategy articulation) and enabling systematic insertion of Kelly Step 4 basis material (T5b).

Observation For g with $type(g) = \mathsf{Goal}$:

$$Obs_{T7}(g) \iff HasSupportChildren(g) \wedge \neg HasStrategyChild(g)$$

where $HasSupportChildren(g)$ holds when $kids(g)$ contains at least one supporter node (e.g., Goal, Evidence, Context, Justification, ModuleRef), and $HasStrategyChild(g)$ holds when $kids(g)$ contains a direct child s with $type(s) = \mathsf{Strategy}$.

Rewrite Introduce a fresh strategy node s:

$$s = \langle \mathsf{Strategy}, id_s, label_s, body_s, kids_s \rangle$$

and rewire:

$$kids_s := kids(g)$$
$$kids(g) := \{s\}$$

The identifiers and text of s are generated deterministically:

- id_s is derived from $id(g)$ and the next available suffix letter $(a, b, c, \dots)$,
- $label_s$ is a stable placeholder strategy label,
- $body_s$ is generated using deterministic templates and begins with an `AUTHOR CHECK:` prefix to indicate required author review.

Effect Provides a uniform structural place for strategy information under each non-leaf goal, reduces Kelly Step 3 deficits, and enables local, deterministic insertion of Step 4 strategy basis material via T5b.

11.10 Deterministic Candidate Generation

Each transformation produces candidate structures:

$$cand(T_i, target, participants, features, rationale)$$

Candidate selection is rule-based; rewrite requires explicit invocation.

11.11 Implementation Architecture

For each T_i:

- `observe_t`i`(Unit, Candidates)`
- `apply_t`i`(Unit, Candidate, NewUnit)`

Recognition is separated from rewrite to maintain determinism and auditability.

11.12 Ordering Strategy

Recommended implementation order:

$$T1 \to T2 \to T5 \to T4 \to T3$$

This ordering minimizes semantic risk and maximizes structural clarity.

11.13 Correctness Constraints

All transformations must preserve:

- Structural well-formedness,
- Identifier stability,
- Sibling ordering (except at rewrite locus),
- LF-only formatting,
- Minimal-diff discipline.

11.14 Summary

The transformation calculus formalizes assurance-case refinement as:

$$U_0 \xrightarrow{T_{i_1}} U_1 \xrightarrow{T_{i_2}} \dots \xrightarrow{T_{i_k}} U_k$$

where each T_i is deterministic, local, and auditable. This provides the foundation for automated refinement within the ACO Workbench while preserving author intent and alignment with Kelly's 6-Step Method.

12 Deterministic Candidate Ranking and Body Text Synthesis

Several transformations (notably T1, T2, and T7) require the Workbench to introduce or relocate natural-language body text. The current implementation distinguishes clearly between:

1. **Deterministic structural rewrites** of the ACO tree (node insertion, deletion, re-parenting, and indentation-managed rewriting), and
2. **Deterministic body-text synthesis** used to populate the bodies of newly introduced nodes in a conservative, review-oriented manner.

The current approach is designed to be usable without LLM support while maintaining minimal-diff discipline and explicit author control.

12.1 Observe–Rank–Transform workflow

The ACO Workbench follows a three-stage workflow:

1. **Observe** — for each transformation T_i, a deterministic observation predicate generates a set of candidates.

2. **Rank** — candidates are scored using a simple Kelly-inspired deficit vector and a churn estimate to prioritize the next refinement step.

3. **Transform** — a selected candidate is rewritten mechanically, preserving all lines outside the rewrite locus byte-for-byte.

This separation ensures that recommendation and mutation are not conflated and supports tool-like behavior.

12.2 Kelly deficit scoring

Candidate ranking uses a small vector of deficits aligned to Kelly's method:

- $k2$ — missing or weak claim basis (Step 2),
- $k3$ — missing or implicit strategy articulation (Step 3),
- $k4$ — missing or implicit strategy basis (Step 4),
- *churn* — estimated disruption due to rewrite scope.

Scores are used to prioritize remediation effort, not to judge overall case quality.

12.3 `AUTHOR CHECK:` convention and conservative synthesis

All deterministically generated or relocated prose is explicitly marked as requiring author review using the prefix:

```
AUTHOR CHECK:
```

This convention is used in two ways:

1. **Placeholders** — when the Workbench cannot confidently synthesize a dischargeable claim or strategy explanation, it emits a generic author-check prompt.

2. **Rule-based synthesis** — when recognizable patterns are present in the surrounding goal/evidence text, the Workbench emits conservative sentences derived from that text (e.g., extracting a subject from a parent goal and predicates from evidence bodies).

The purpose of the convention is to preserve author intent and prevent silent semantic strengthening: the Workbench may restructure and suggest, but the author remains responsible for accepting or revising the newly introduced text.

12.4 Relationship to bounded LLM refactoring

The deterministic synthesis described here is sufficient for structural demonstration and initial review workflows. However, empirical use on the RPM case shows that higher-quality prose often requires disassembly and reassembly of evidence text beyond simple templates. The next section therefore introduces bounded LLM-assisted semantic refactoring as an optional enhancement that improves rhetorical quality while preserving structural determinism and auditability.

13 Bounded LLM-Assisted Semantic Refactoring

13.1 Rationale

Structural transformations (T1–T7) operate deterministically over the Assurance Case Outline (ACO) tree. However, several transformations (notably T1, T2, and T5) require non-trivial rewriting of natural-language content when splitting, relocating, or introducing nodes.

Empirical experimentation on the Remote Patient Monitoring (RPM) assurance case demonstrated that purely templated rewriting often produces structurally correct but rhetorically weak results. In particular, effective transformation requires:

- Disassembling composite evidence prose into semantically distinct roles (artifact, method, result, adequacy),
- Selecting fragments appropriate to new node types,
- Reassembling fragments coherently under new rhetorical constraints,
- Explicitly identifying gaps rather than silently inferring missing content.

To support this requirement without sacrificing determinism, we introduce *Bounded LLM-Assisted Semantic Refactoring*.

13.2 Architectural Principle

Structural transformation and semantic rewriting are strictly separated:

1. **Phase A (Deterministic Structural Rewrite)** The ACO Workbench performs tree rewrites mechanically and deterministically. Node identities, relationships, and insertion points are fully defined by transformation rules.
2. **Phase B (Bounded Semantic Refactoring)** An LLM may be invoked to rewrite designated node bodies, subject to explicit structural and semantic constraints.

The LLM is never permitted to:

- Create, delete, or re-type nodes,

- Modify parent/child relationships,
- Introduce new factual claims not supported by the original text,
- Alter identifiers or metadata.

The transformation logic remains authoritative; the LLM assists only with controlled semantic refactoring.

13.3 Formal Refactoring Contract

Let U be the original unit and U' the result of a deterministic structural rewrite. Let $S \subseteq N'$ be the set of nodes whose bodies may be rewritten.

A bounded semantic refactoring is defined as:

$$Refine : (U, U', S, P) \to U''$$

where:

- S is the explicitly enumerated set of editable nodes,
- P is a fixed prompt template version,
- U'' differs from U' only in $body(n)$ for $n \in S$.

$$\forall n \notin S, \quad body_{U''}(n) = body_{U'}(n)$$

$$\forall n \in S, \quad type(n), id(n), kids(n), parent(n) \text{ remain unchanged}$$

13.4 Traceability Constraint

For each rewritten node $n \in S$, the LLM must return:

- $body_{new}(n)$,
- $trace_map(n)$: a mapping of each newly generated sentence to supporting fragments from $body_{original}(n_{source})$,
- $gaps(n)$: explicit statements of unsupported assumptions or missing criteria.

Formally:

$$trace_map(n) = \{(s_i, f_i) \mid s_i \in Sentences(body_{new}(n)),\ f_i \subseteq body_{original}\}$$

No sentence may appear in $body_{new}(n)$ without either:

- A cited supporting fragment f_i, or
- An explicit designation as a structured placeholder.

13.5 Transformation-Specific Contracts

T1 (Evidence Node Slimming) Editable nodes:

$$S = \{e', c, j\}$$

The LLM must:

- Extract artifact anchor text into e',
- Move method/process text into c,
- Move adequacy rationale into j,
- Preserve explicit artifact identifiers (e.g., xref).

T2 (Evidence-Discharging Goal Introduction) Editable nodes:

$$S = \{g'\}$$

The LLM must:

- Formulate a dischargeable intermediate goal claim,
- Ensure the claim is strictly entailed by the associated evidence,
- Avoid strengthening beyond the original evidence.

T5 (Basis Articulation) Editable nodes:

$$S = \{c_{basis}, j_{basis}\}$$

The LLM must:

- Explicitly articulate scope, definitions, and assumptions (Step 2),
- Articulate coverage and sufficiency rationale (Step 4),
- Identify unresolved completeness gaps where appropriate.

13.6 Determinism and Versioning

Each refactoring invocation records:

- Original body hash,
- Structural transformation identifier,
- Prompt template version P,
- LLM model identifier,
- Output body hash,
- Traceability map.

This ensures that semantic rewriting remains auditable and reproducible.

13.7 Fallback Mode

If LLM refinement is disabled, deterministic template-based bodies are generated. These provide structural correctness but may lack rhetorical clarity. This mode ensures that transformations remain executable in environments without LLM support.

13.8 Separation of Concerns

This architecture preserves:

- Structural determinism (ACO Workbench),
- Method discipline (Kelly 6-Step Assistant),
- Semantic clarity (bounded LLM refactoring),
- Provenance authority (REPOSITORY),
- Evaluator stability (CAP snapshot).

The LLM is thus treated not as an autonomous author, but as a constrained semantic refactoring engine operating within formally defined transformation boundaries

14 Action Layer and Constructor Basis

The transformation framework described above is now complemented by an *action layer* that treats both forward-building constructors and structure-improving rewrites as explicit, inspectable workbench actions. This layer is implemented separately from `aco_transforms` in order to keep the transformation module focused on concrete tree rewrites while allowing the emerging action theory, candidate generation, ranking, and diagnostic logic to evolve in a cleanly separated module.

14.1 Canonical action representation

The common currency of the action layer is the fully specified action term:

```
action(Name, Kind, Target, Params)
```

where `Name` identifies the operation, `Kind` distinguishes constructor from rewrite (and potentially other kinds in the future), `Target` identifies the locus of application (the form of which depends upon the operation), and `Params` contains operation-specific parameters.

The chosen ordering gives the action name first position for efficient indexing and natural dispatch, while retaining the full action object as the semantic unit that is observed, ranked, selected, applied, and reported. Keeping `Kind` within the action term avoids the maintenance burden and drift risk of a separate `action_kind/2` table.

14.2 Candidates, observation, and ranking

Workbench recommendation logic operates over explicit candidates of the form:

```
candidate(Action, Score, Rationale, Diagnostics)
```

Observation functions generate a set of currently applicable constructor and rewrite candidates for a given ACO configuration. Ranking functions then order those candidates using pluggable evaluation policies.

The current design deliberately supports both deterministic evaluation functions and future intelligent evaluation and advisory capabilities. This approach provides a stable deterministic baseline behavior that can serve as a primary or fallback position while allowing richer alternatives to be introduced without changing the action or candidate vocabulary.

14.3 Constructor basis for assurance case construction

A central concept in the ACO Workbench is the notion of a basis set of constructors that makes it possible to build any well-formed assurance case configuration from an initially empty configuration. The intended progression is:

1. Start from an empty case configuration.

2. Apply `add_goal` to create the first top-level goal.

3. Enable further local constructors such as `add_strategy`, `add_subgoal`, `add_context`, `add_justification`, `add_assumption`, and `add_evidence`.

4. Once sufficient structure exists, enable rewrite-style transformations such as T1–T7 to improve or reorganize the argument structure.

Primitive constructors are intentionally simple. For example, `add_strategy` should add a strategy node but not also restructure the subtree beneath it as such a structural rewrite is handled separately by transform T7. Likewise, `add_evidence` should attach evidence without also normalizing or introducing a discharge goal; such improvement remains the role of transform T1 or T2. This separation preserves a clean distinction between construction, improvement, and guidance based on methodology and metrics.

14.4 Targets and goal nesting

The target vocabulary retains both direct node targets and pairwise targets. In particular, the `pair/2` target form, first used in conjunction with the T2 transform, remains important for operations that conceptually act over a relation, insertion boundary or the roots of parallel structures rather than a single node.

Direct goal nesting is also retained as a valid construction step through a first-class `add_subgoal` constructor. This is methodologically important because an author will often decompose a goal into child goals before deciding to introduce an explicit strategy node between the parent and those subgoals. T7 then provides a principled structural rewrite for that later clarifying step.

14.5 Diagnostics and reporting

The action layer also provides a natural locus for a more systematic diagnostic messaging architecture. Rather than limiting diagnostics to parser or validator messages, the Workbench can report:

- which actions are currently applicable,
- why a candidate is applicable,
- which deficits or opportunities motivated its recommendation,
- what structural edits were actually performed, and
- which invariants were revalidated after application.

This diagnostic vocabulary still needs to be fleshed out in concrete schema form, but the architectural place for doing so is now clear.

14.6 Relation to GSN and APL completeness

The immediate action-layer work is being undertaken in parallel with a broader objective: ACO must be able to express, either directly or through a combination of primitive constructs, the set of APL constructs that are currently valid and useful in the O-ETB pipeline. This will require revisiting the relationship among:

- GSN-derived structures,
- ACO notation and constructors,
- ACO-to-APL translation, and
- the existing pattern corpus under `KB/PATTERNS` and the earlier `gsnml` conversions to APL.

In the near term this includes support for iterator-like and other structural abstraction mechanisms that are already meaningful at the APL level but not yet well represented in ACO. Longer term it should also reconcile ACO development with GSN structural abstraction mechanisms and the modular/pattern-oriented representations already present in the O-ETB knowledge base.

15 Iterator Validation Boundaries and Extensible Iterand Semantics

This section records an important architectural distinction for iterator-bearing constructs in ACO and APL, so that subsequent implementation work does not collapse several different concerns into a single mechanism.

15.1 Motivation

ACO is being extended toward fuller practical coverage of the APL constructs already used in O-ETB patterns and assurance case generation. A particularly important class of constructs is the family of iterator-bearing strategy forms used to express repeated argument structure over a collection of entities, flows, hazards, modes, transitions, and similar designators.

Examples already present in the existing O-ETB/APL ecosystem include iterator designators such as:

```
nodes(S), subjects(Policy), processes(Policy), ss_flows(Policy),
ipc_flows(Policy), hazards(Policy), modes(Subject), transitions(Subject),
list(Values), expand(X)
```

These designators raise a subtle issue of architectural responsibility. The ACO processor must be able to parse and translate an iterator-bearing construct reliably, but it need not itself *execute* the iterator against a model, tool output, or knowledge base.

15.2 Three distinct notions of well-formedness

The design should explicitly distinguish the following three levels:

1. **ACO-level syntactic well-formedness**
   At this level, the concern is whether the iterator-bearing ACO syntax is sufficiently definite to be parsed and translated deterministically. In particular, the `<iterand>` field must be syntactically well-formed enough to be recognized as a term-valued designator rather than arbitrary prose.

2. **APL-level semantic well-formedness**
   At this level, the concern is whether the parsed iterand term denotes a known and properly formed APL iterator/designator. This is stronger than mere syntactic well-formedness, but weaker than actual execution. For example, `hazards(Policy)` may be recognized as a valid iterator designator form even if the current environment does not yet know how to enumerate the hazards of the given policy.

3. **Instantiation-level operational realizability**
   At this level, the concern is whether the iterator can actually be evaluated in the current environment, using the available models, tools, knowledge bases, or cached analysis artefacts. This is where actual expansion occurs.

These three levels should not be conflated.

15.3 Design consequence for ACO

The current design decision is that the `<iterand>` field in an iterator-bearing ACO construct may be *semantically opaque* to the ACO processor, but it must still be *syntactically definite* enough to translate reliably.

Accordingly:

- the ACO processor does *not* need to execute iterator designators;
- the ACO processor *does* need to parse them into a term-like representation deterministically;
- the resulting ACO $\rightarrow$ APL translation must be semantically well-defined;
- execution or expansion of the iterator remains a downstream concern.

This means that a term-valued representation of `<iterand>` is acceptable and desirable.

15.4 Architectural implication: an O-ETB validation API

To support the distinction between ACO-level parsing and downstream execution, O-ETB should expose an API that allows the ACO layer to ask whether a parsed iterand term is a known and properly formed APL iterator/designator.

Conceptually, the interface would support a question of the form:

```
is_this_a_known_and_well_formed_APL_iterator_designator(Term)?
```

or, more explicitly,

```
apl_iterand_well_formed(+Term,-Diagnostic)
```

The purpose of such an API is *validation*, not execution. In particular, it would allow the ACO processor to reject or warn about malformed iterator designators without taking on responsibility for enumerating their values.

15.5 Extensible provider architecture

A more general issue also arises. Many useful iterands do not refer merely to fixed built-in collections. Rather, they refer to:

- special interfaces into `KB/MODELS`,
- outputs of analysis tools or model transformations,
- derived artefacts such as hazards, modes, transitions, or flows, or
- other project-specific sources of information about the system under scrutiny.

This suggests that iterator execution should eventually be mediated by an extensible provider architecture rather than being hard-wired monolithically inside a single instantiation module.

The intended division of responsibilities is therefore:

- **ACO layer:** parse iterator-bearing syntax and normalize the iterand into a term;
- **APL/O-ETB validation layer:** determine whether the term is a known and properly formed iterator/designator;
- **Instantiation/provider layer:** evaluate the iterator against the relevant model, tool, repository, or derived artefact source.

This architecture allows the assurance engineer, rather than necessarily the assurance case author, to configure and extend the available iterator primitives to suit the artefacts and tools available for a given project.

15.6 Implementation guidance

The implementation should preserve the following principles:

- ACO should remain readable and author-oriented; it should not collapse into raw APL notation.

- Iterator-bearing syntax should lower deterministically into an existing APL representation.
- Validation should distinguish clearly among:
  - syntactically well-formed iterators,
  - semantically well-formed APL iterator designators, and
  - operationally available iterator providers.
- The eventual iterator/provider mechanism should be extensible and configurable by the assurance engineer.

15.7 Status

This section records a design decision and architectural direction. It does not imply that the current implementation already provides the full validation API or provider architecture described above. Those remain explicit follow-on design and implementation tasks.